分析分两步进行。先用“objdump-d--disassemble=函数名”把目标函数导出到文件，通读控制流，找出分支、循环和比较，推测出计算关系；再在 gdb 中下断点，停在关键的比较指令之前，用 x、p 查看寄存器和内存，确认推测的值与程序实际算出的值一致。离线分析确定"算什么"，在线调试确认"算得对不对"，两者交替使用。

每关需要的整数个数由关卡函数开头传给 valid_u32_fields 的第二个参数决定，输入格式由随后的 sscanf 格式串决定。解法中的中间量统一使用 uint32_t，与汇编中 32 位寄存器的回绕语义保持一致。

第一关 p1_compute 把 challenge 拆成高 32 位、低 32 位和右移 16 位三份，依次调用 p1_step0 、 p1_step1 、 p1_step2 ， 每 个 函 数 只 有 两 三 条 指 令 。 step0 计 算 (hi^0x3a76)+2*challenge，step1 计算(mid+0xb483)^hi，step2 计算 3*r1^(lo+0x5e53)。函数末尾的 xor%ecx,%eax 把 step0 的结果并进返回值；ecx 保存了它，而三个子函数都没有改写过 ecx。第二关第一个是(challenge>>38)&0x1f，其余五个由循环产生：第 i 个为((challenge>>(5i))&0x3f)+i，再与 3*x[i-1]异或。循环中用 lea 先算出 5*i 放入 ecx，因为 shr 的移位次数必须放在 cl 中；3*x[i-1]是在 esi 被当前输入覆盖之前计算的，因此使用的是上一个数。第三关第一个等于 challenge&7，作为下标；第二个由 p3_dispatch 按跳转表调用对应的处理函数，参数为(challenge>>3)^(challenge>>32)。跳转表在 0x405820，八项依次对应 case_1、case_2、case_0、case_4、case_5、case_6、case_3、case_7，与 case 编号的书写顺序不一致。我最初按下标递增对应 case 编号实现，十个公开 seed 只通过两个，后来用 x/8gx 0x405820 读出表内容才改对。第四关 a=challenge&0x7f 是二分查找的 目 标 ， 区 间 为 [0,127] ， k=(challenge>>7)^(challenge>>32) 。 p4_walk 计 算 mid=lo+(hi-lo)/2，命中返回(mid+0x88)^k，向左返回 3*递归^(mid+k)^0x16，向右返回 3*递归^(mid+k)^0x1f。左右异或的常量不同，因此结果与路径有关。实现时没有另写递归函数，而是在下行时记录每层的 mid+k 和方向，命中后按相反顺序折回，与递归的求值顺序一致。第五关节点结构为{next,f8,f12}，16 字节，12 个节点存放在.rodata 的 0x405ba0 到 0x405c50。challenge 的低 3 位选择入口（入口表在 0x405b60），第 3、4 位决定沿 next 走 的 步 数 （ 0 到 3 ） ， 第 6 位 起 参 与 初 值 异 或 。 计 算 过 程 为 ： 初 值 (challenge>>6)^(challenge>>32)^f12^0xb500，加上 f8；之后每走一步异或该节点的 f12、加上它的 f8；最后输出累加值和最后一个节点的 f8。需要注意 xor$0xb5,%ah 修改的是第 8 到 15 位，等价于异或 0xb500。节点数据是程序的静态数据，对所有 challenge 都相同，因此直接写进解法。第六关函数指针表在 0x405e00,共有两次间接调用:第一次以 challenge&7 为 下 标 ， 参 数 为 (challenge>>6)^(challenge>>32) ； 结 果 异 或 0xef67 后 ， 以 (challenge>>3)&7 为下标再调用一次，最后加 0xf232。表中有重复项，下标 0 和 3 都是 p6_op_4，1 和 4 都是 p6_op_1。p6_op_4 是循环右移 1 位，p6_op_5 是循环左移 7 位再异或 0xada3；C 语言没有循环移位运算符，用(x>>1)|(x<<31)和(x<<7)|(x>>25)实现。Honer 关第七行的长度必须在 36 到 163 字节之间。secret_gate 把整行复制进一个 208 字节的栈缓冲区，然后用 cmpl$0x4e45504f,-0xb0(%rbp)比较缓冲区第 32 字节处的四个字节，该立即数按小端解释为 OPEN。因此前 32 字节是填充，第 32 到 35 字节必须是 OPEN，第 36 字节起是三个整数，由 secret_phase 用%u%u%u 解析。

三个数的计算如下：

x0=context&3;

c1=(challenge>>32)^challenge,

c2=(context>>32)^context;

x1=mix(c1^c2)^(0x7cdd*x0^(context>>16)),

其中 mix(x)=(7x+0x7cdd)^0xf4f5；
x2=x1^(0xf4f5+c2+x0+(c1>>3))。
在 gdb 中用 x/40cb$rbp-0xd0 可以看到缓冲区的前 32 字节、紧随其后的 OPEN 和数字，与比较指令中的偏移一致。
六关的公开测试全部通过（60/60），Honor 为 10/10。此外用 seed 43 完整运行了六关和 Honor，seed 43 不在公开 seed 列表中，七关均输出 defused。
第 3、4、5、6 关的分支数量有限，用--challenge 指定 challenge 做了穷举：第 3 关 8 项，第 4 关 512 项（128 个 a 值乘 4 个 k），第 5 关 64 项，第 6 关 128 项，全部通过。
汇编的解读、公式的转写和报告措辞使用了 AI 助手(DSH)，所有公式都在本机用--challenge 和 public_test.py 重新计算验证，文中引用的汇编和内存数据来自自己运行的 objdump 与 gdb。调试时只使用 break、run、continue、stepi、nexti、finish、x、print、bt 等观察类命令，以及 set pagination off 和保存观察值的便利变量，没有使用 set 修改程序的寄存器、内存或控制流。
注：剧情可以。

```
nanming@Nanming:~/ics/ics-2026-bomblab$ python3 public_test.py
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

```
Breakpoint 1, 0x0000000000402720 in secret_gate ()
0x7fffffffd410: "0123456789abcdef0123456789abcdefOPEN 0 1174261839 1575246609"
$1 = 16296930356100118074
$2 = 11617819588523061488
Breakpoint 2 at 0x40278b

Breakpoint 2, 0x000000000040278b in secret_gate ()
0x7fffffffd190: 48 '0'  49 '1'  50 '2'  51 '3'  52 '4'  53 '5'  54 '6'  55 '7'
0x7fffffffd198: 56 '8'  57 '9'  97 'a'  98 'b'  99 'c'  100 'd' 101 'e' 102 'f'
0x7fffffffd1a0: 48 '0'  49 '1'  50 '2'  51 '3'  52 '4'  53 '5'  54 '6'  55 '7'
0x7fffffffd1a8: 56 '8'  57 '9'  97 'a'  98 'b'  99 'c'  100 'd' 101 'e' 102 'f'
0x7fffffffd1b0: 79 'O'  80 'P'  69 'E'  78 'N'  32 ' '  48 '0'  32 ' '  49 '1'
```

后面是这些年断断续续的更新。
一张北方的雪照，一次换了方向的项目，一段拖了很久才修好的代码。
sky 去了北方，许遥留在这里。他们各自忙起来，
回复有时隔了好几天，也会聊新认识的朋友。
有一阵，许遥没再继续原来的项目。后来 sky 也换了工作。
这座城倒还在。谁回来，就顺手添一点东西。

观测台的灯是 sky 修的，两把椅子是许遥后来搬来的。
最后一张便签贴在灯座旁边：

‘下次回来，先坐一会儿。’

林夏拉开其中一把椅子。你们坐了一会儿，没有再翻记录。
街上亮起今天的新灯。

离开观测台时，你想起自己收藏了很久的一份项目介绍，
把它放进明天的待办。林夏也在自己的本子上记了几行。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

谢谢你远道而来，与我相遇。

晚安。

—— sky